

Homework 2

600.482/682 Deep Learning

Fall 2025

Due Wednesday, Sep 17th, 2025, 11:59 pm EST

Submission Instructions. Please submit the following **two items** to Gradescope (entry code VWJRB3):

1. Your report (LaTeX generated PDF) named `homework2-report.pdf`.
2. A Zip file containing: (1) Your Python Jupyter Notebook (`homework2-notebook.ipynb`) and (2) an exported PDF of the notebook (with all cell outputs) to `homework2-notebook.pdf`.
 - * On Google Colab, you may use “File” - “Print” - Save as PDF.
 - * On Visual Studio, you may use “Jupyter: Export to PDF” in the Command palette.

Problems

1. The goal of this problem is to minimize a function given a certain input using gradient descent by breaking down the overall function into smaller components via a computational graph. The function is defined as:

$$\begin{aligned} f(x_1, x_2, w_1, w_2) &= \sigma(w_1x_1 + w_2x_2) + 0.5(w_1^2 + w_2^2) \\ &= \frac{1}{1 + e^{-(w_1x_1 + w_2x_2)}} + 0.5(w_1^2 + w_2^2). \end{aligned}$$

- (a) Please calculate $\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}$. When computing the derivative for the sigmoid function $\sigma(x)$, please use sigmoid function $\sigma(x)$ in your expression.

$$\text{Let } z = w_1x_1 + w_2x_2, \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = -(1 + e^{-z})^{-2}(-e^{-z})$$

$$= \frac{e^{-z}}{(1 + e^{-z})^2}$$

$$= \left(\frac{1}{1 + e^{-z}}\right)\left(\frac{e^{-z}}{1 + e^{-z}}\right)$$

$$= \left(\frac{1}{1 + e^{-z}}\right)\left(\frac{1 + e^{-z} - 1}{1 + e^{-z}}\right)$$

$$= \left(\frac{1}{1 + e^{-z}}\right)\left(\frac{1 + e^{-z}}{1 + e^{-z}} - \frac{1}{1 + e^{-z}}\right)$$

$$= \sigma(z) \cdot \left(1 - \frac{1}{1 + e^{-z}}\right)$$

$$= \sigma(z) \cdot (1 - \sigma(z))$$

$$\frac{\partial f}{\partial x_1} = \sigma'(z) \cdot \frac{\partial}{\partial x_1}(w_1x_1 + w_2x_2) + \frac{\partial}{\partial x_1}0.5(w_1^2 + w_2^2)$$

$$= \sigma(z)(1 - \sigma(z))w_1 + 0$$

$$= \sigma(z)(1 - \sigma(z))w_1$$

$$\begin{aligned}
\frac{\partial f}{\partial x_2} &= \sigma'(z) \cdot \frac{\partial}{\partial x_2}(w_1 x_1 + w_2 x_2) + \frac{\partial}{\partial x_2} 0.5(w_1^2 + w_2^2) \\
&= \sigma(z)(1 - \sigma(z))w_2 + 0 \\
&= \sigma(z)(1 - \sigma(z))w_2 \\
\frac{\partial f}{\partial w_1} &= \sigma'(z) \cdot \frac{\partial}{\partial w_1}(w_1 x_1 + w_2 x_2) + \frac{\partial}{\partial w_1} 0.5(w_1^2 + w_2^2) \\
&= \sigma(z)(1 - \sigma(z))x_1 + w_1 \\
\frac{\partial f}{\partial w_2} &= \sigma'(z) \cdot \frac{\partial}{\partial w_2}(w_1 x_1 + w_2 x_2) + \frac{\partial}{\partial w_2} 0.5(w_1^2 + w_2^2) \\
&= \sigma(z)(1 - \sigma(z))x_2 + w_2
\end{aligned}$$

- (b) Start with the following initialization: $w_1 = 0.2, w_2 = 0.4, x_1 = -0.4, x_2 = 0.5$, draw the computational graph. Please use backpropagation as we did in class. You may use the sigmoid function $\sigma(x)$ as a node in the graph.

You can draw the graph on paper and insert a photo into your report.

The goal is for you to practice working with computational graphs.

You must include the intermediate values in both forward and backward passes in your graph.

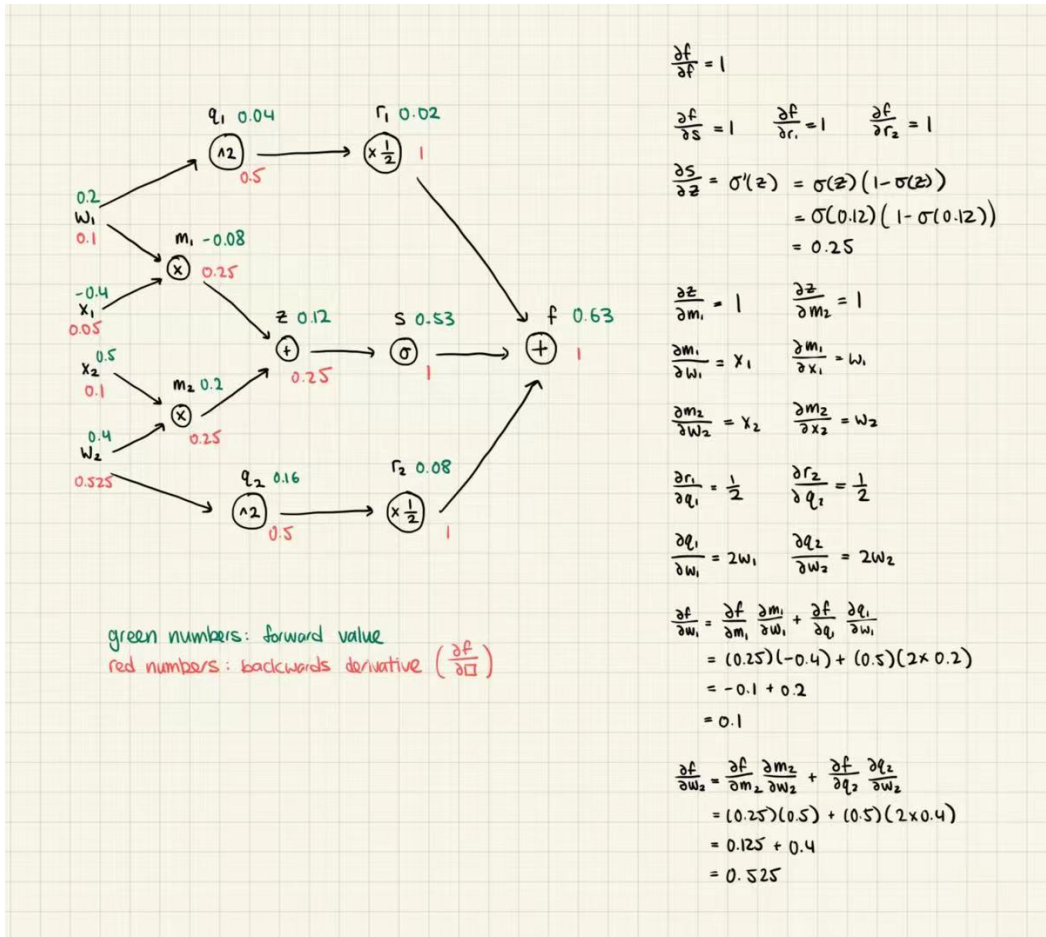


Figure 1: Computational graph of $f(x_1, x_2, w_1, w_2)$.

- (c) John would like to train a binary classification model that classifies whether a particular fruit is an apple or not based on two features: its color (x_1) and its roundness (x_2). w_1 and w_2 are the weights he gives to the two features. John tries to use the function $f(x_1, x_2, w_1, w_2)$ as the loss function for a single data point in his optimization problem.

Will this formulation of loss function work? Briefly explain why or why not.

No, this fomulation of loss function would not work because there is no bias term nor a comparison to the true label (i.e. 1 for apple, 0 for not apple). For it to be a proper loss function, after getting the linear score $z = w_1x_1 + w_2x_2 + b$ (bias term) and applying the sigma function $\hat{y} = \sigma(z)$, we would need to compare $\hat{y}$ to y (true label).

2. Doing the computations above is a lot of work, already for such a simple function as in Problem 1. Clearly, it will be desirable to automate the forward and backward propagation process. In contemporary frameworks, such as TensorFlow and PyTorch, backpropagation is handled automatically for all standard operations, which is very convenient. This feature is commonly referred to as “AutoGrad”. It becomes possible due to one of the key observations we learned in class: difficult expressions can be broken down into simple sub-problems, the analytic gradients of which can be synthesized via chain rule. In this problem, you will implement your own “AutoGrad” structure.
 - (a) Consider the Jupyter Notebook we provide (*Homework2.ipynb*, which can be downloaded from https://piazza.com/class_profile/get_resource/mdnhowckj2f5di/mf451vw4w4q1ci) and follow the instructions therein to implement the missing operations and functionality. To use Jupyter Notebook, we highly recommend uploading the *Homework2.ipynb* file to Google Colab (<https://colab.research.google.com/>), where you may edit and run your notebook online. If that option is not feasible for you, you can create a local Python 3.6 (or higher version) environment. The only required external dependencies are `numpy`, `matplotlib`, and `jupyter` which you can install using the package manager of your choosing (e.g. `pip`, `conda`).
 - (b) Use the function and output derived in Problem 1 to test whether your “AutoGrad” structure works as intended.

Attach a screenshot of your notebook’s printed intermediate values for this test case (both during the forward and backward pass) to your report.

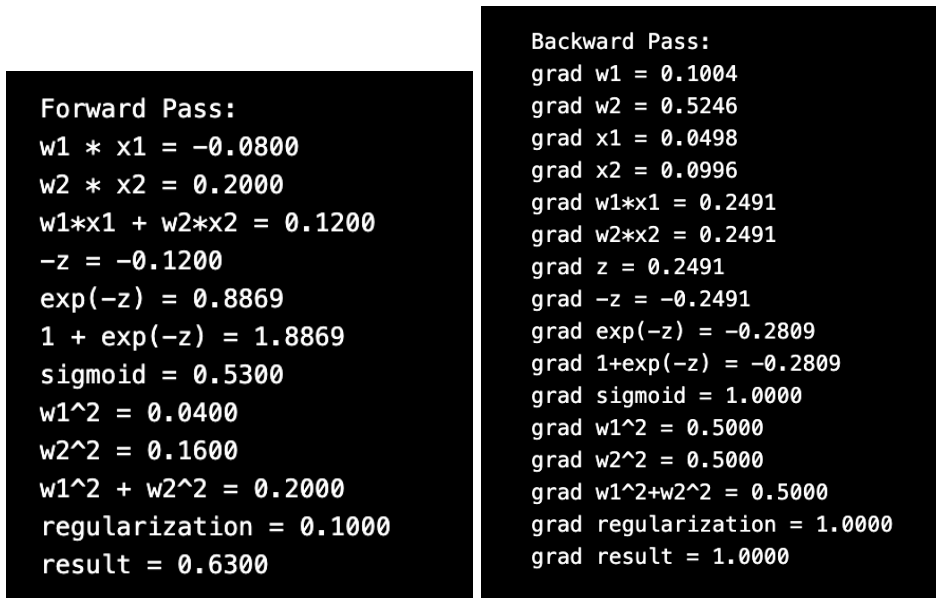


Figure 2: Forward and backward pass intermediate values

3. Your goal in this exercise is to implement a linear classifier using the AutoGrad class you implemented in Problem 2, train it on a 2-dimensional toy dataset, and show the results tested on the **same** dataset. Recall that we formulate a linear classifier as $f(x, W, b) = Wx + b$. To quantify the model’s performance, you will pass its output through a softmax function and then use Cross Entropy loss to measure agreement with the target output.

Write a Python program based on your AutoGrad structure that iteratively computes a small step in the direction of the negative gradient. The dataset (HW2_Q3_dataset) can be downloaded from https://piazza.com/class_profile/get_resource/mdnhowckj2f5di/mf44x67wv1r47b. **Please go through the README.txt carefully before you start.** Initialize your optimization with all elements of W being small random numbers and $b = 0$. As shown in the visualization of the data provided in the link above, you can reasonably expect to see your algorithm converge to a solution that linearly separates the given data samples.

- (a) Please plot the following and attach them in the report:
- Cross Entropy loss with respect to training iterations (loss curve)
 - Training accuracy with respect to training iterations (accuracy curve)
 - Visualization of the ground truth labels and your model's decision boundaries (decision boundary visualization)

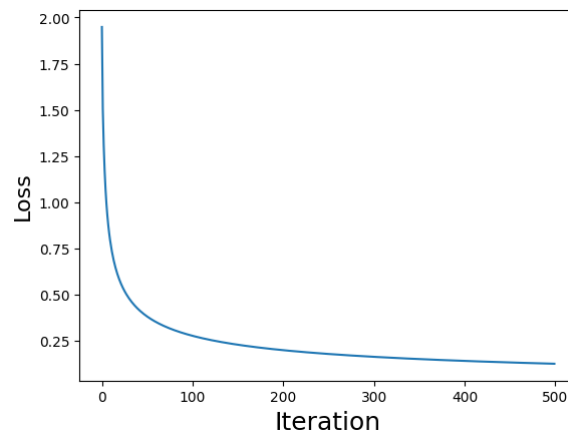


Figure 3: Cross Entropy loss with respect to training iterations (loss curve)

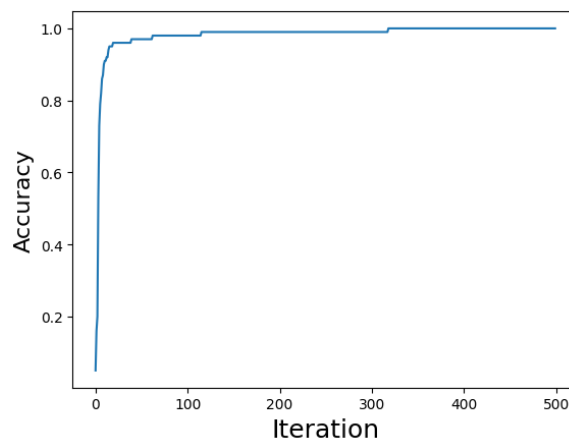


Figure 4: Training accuracy with respect to training iterations (accuracy curve)

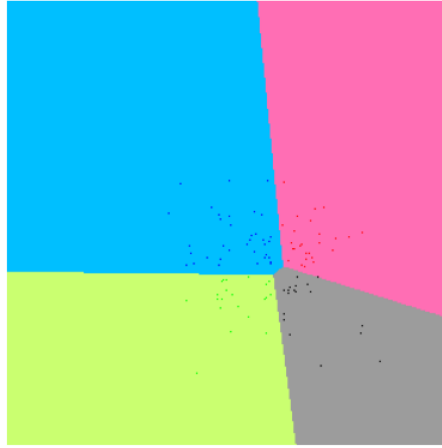


Figure 5: Visualization of the ground truth labels and your model's decision boundaries (decision boundary visualization)

- (b) Report your final classification accuracy. Briefly discuss your result based on the visualization of data distribution.

The final classification accuracy is 1, which means that the model has classified all of the points with the correct labels. We can also see from the above visualization that all the dots are in their corresponding colored background.

4. In this problem, we would like you to compare the performance of single-layer and multi-layer classifiers on a different dataset, again using your AutoGrad structure.

- (a) Please use HW2_Q4_dataset (https://piazza.com/class_profile/get_resource/mdnhowckj2f5di/mf44x33xayp450) for this question. This data is two-dimensional. Create another linear model (same as Problem 3) for this dataset and run your classification training.

Report your final classification accuracy. As in Problem 3, please attach plots of the i) loss curve, ii) accuracy curve, and iii) decision boundary visualization. Is this dataset linearly separable?

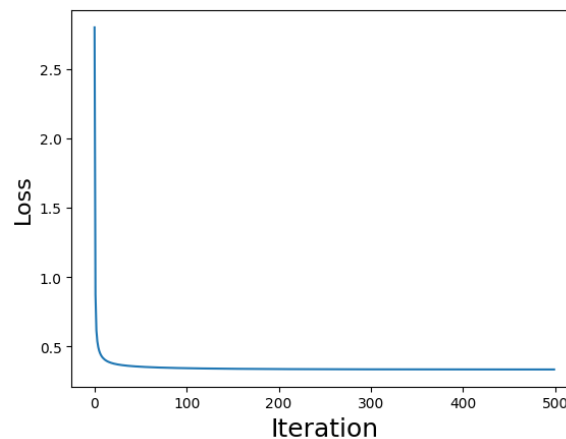


Figure 6: Cross Entropy loss with respect to training iterations (loss curve)

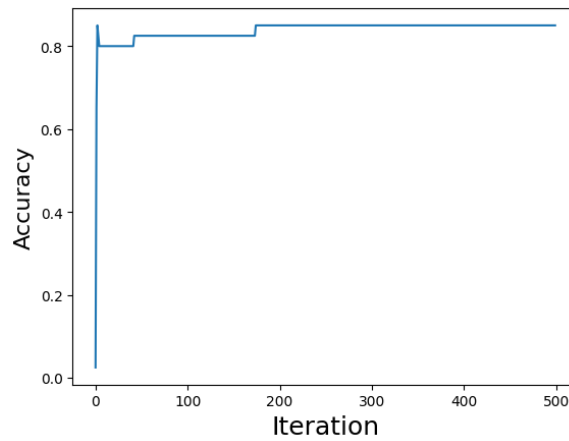


Figure 7: Training accuracy with respect to training iterations (accuracy curve)

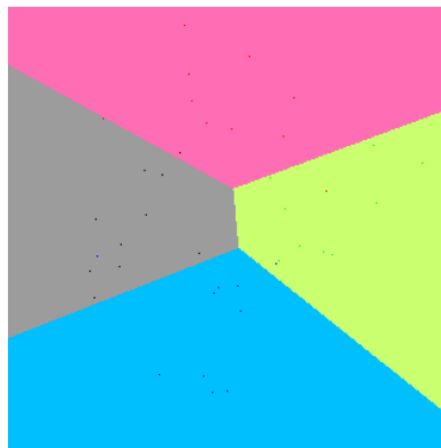


Figure 8: Visualization of the ground truth labels and your model's decision boundaries (decision boundary visualization)

The final decision accuracy is 0.85. And no, this dataset is not linearly separable. The accuracy not equaling 1 indicates that a linear classifier is fundamentally incapable of achieving perfect separation of this dataset. From the graph, the classes appear to be not clustered in single, distinct regions but are instead dispersed and overlap with one another.

(b) Now add one more layer of perceptions.

* Use ReLU as activation functions for the first hidden layer.

** Connect outputs with a softmax function. Initialize your optimization procedure with all elements of W being small random numbers and $b = 0$. Use Cross Entropy loss. Train this model on the same two-dimensional dataset in (a).

Report your classification accuracy. Attach plots of the i) loss curve, ii) accuracy curve, and iii) decision boundary visualization in the report.

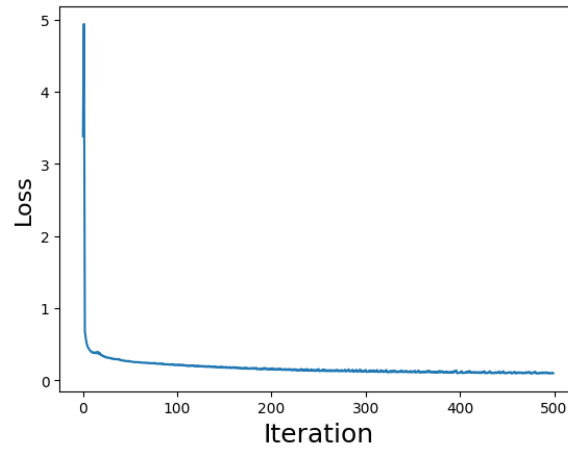


Figure 9: Cross Entropy loss with respect to training iterations (loss curve)

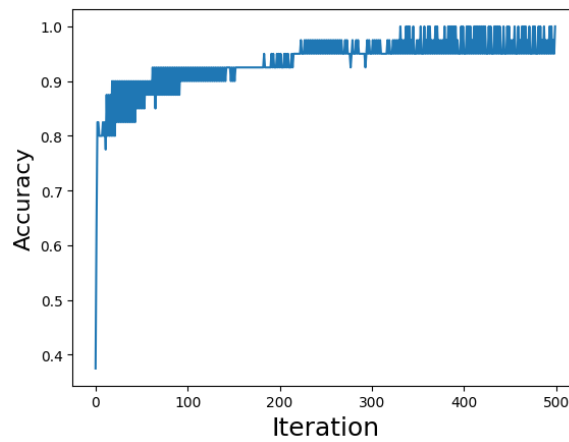


Figure 10: Training accuracy with respect to training iterations (accuracy curve)

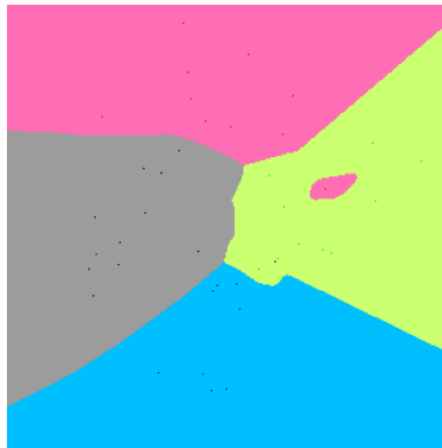


Figure 11: Visualization of the ground truth labels and your model's decision boundaries (decision boundary visualization)

- (c) Compare your results, including the classification accuracy and decision boundary plot, between 4(a) and 4(b). Briefly explain why introducing an additional layer causes these changes.

* If the output of a single layer is $f(x, W_1, b_1)$, then adding another layer basically means that the output is $g(f(x, W_1, b_1), W_2, b_2)$: the second layer uses the first layer's output as its input.

Therefore, the output dimension of f should match the expected input dimension of g . In our case, the output dimension from the ReLU activation in the first layer should match the input dimension of the second layer.

** The ReLU activation function is a nonlinear function defined as:

$$\text{ReLU}(x) = \max(0, x)$$

If we use ReLU as our activation function for the first layer, instead of $f(x, W_1, b_1) = W_1x + b$, we have

$$f(x, W_1, b_1) = \text{ReLU}(W_1x + b) = \max(0, W_1x + b)$$

The non-linearity of the ReLU function adds non-linearity to the model.

Unlike the linear model that stalled at 0.85 accuracy due to its inherent limitations, the Multi-Layer Perceptron (MLP) successfully learned the complex patterns in the non-linearly separable data (final classification accuracy of 1 after oscillating). By using its hidden layers to form decision regions, the MLP adapted to the data's distribution and achieved perfect classification, demonstrating how necessary the non-linear model is for this problem.